

Лабораторная работа №8

**Реализация основных моделей в дискретно-событийном
подходе**

Мария Улитина

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
3.1	Модель экспоненциального роста	8
3.2	Инструменты	9
4	Выполнение лабораторной работы	10
4.1	1. Установка Git	10
4.2	2. Настройка Git	10
4.3	3. Базовая конфигурация Git	10
4.4	4. Создание SSH-ключа	11
4.5	5. Установка Node.js	11
4.6	6. Установка yarn	11
4.7	7. Обновление списка пакетов	11
4.8	8. Установка nodejs и npm	12
4.9	9. Проверка версий и установка npnm	12
4.10	10. Настройка npnm	12
4.11	11. Установка Git Flow	13
4.12	12. Создание структуры рабочего пространства	13
4.13	13. Клонирование репозитория	13
4.14	14. Настройка каталога курса	14
4.15	15. Отправка изменений на GitHub	14
4.16	16. Инициализация Git Flow	14
4.17	17. Проверка веток и отправка	14
4.18	18. Создание первого релиза	15
4.19	19. Установка standard-changelog	15
4.20	20. Создание CHANGELOG	15
4.21	21. Коммит CHANGELOG	15
4.22	22. Установка Julia	16
4.23	23. Запуск Julia	16
4.24	24. Установка DrWatson	16
4.25	25. Инициализация проекта DrWatson	16
4.26	26. Установка пакетов Julia	17
4.27	27. Создание тестового скрипта	17

4.28	28. Запуск тестового скрипта	17
4.29	30. Реализация модели экспоненциального роста	18
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	Установка Git	10
4.2	Настройка имени и email в Git	10
4.3	Базовая конфигурация Git	10
4.4	Создание SSH-ключа	11
4.5	Установка Node.js	11
4.6	Установка yarn	11
4.7	Обновление списка пакетов	12
4.8	Установка nodejs и npm	12
4.9	Проверка версий и установка npnm	12
4.10	Настройка npnm	13
4.11	Установка Git Flow	13
4.12	Создание рабочего пространства	13
4.13	Клонирование репозитория	13
4.14	Настройка каталога курса	14
4.15	Push на GitHub	14
4.16	Инициализация Git Flow	14
4.17	Проверка веток и push	14
4.18	Создание релиза	15
4.19	Установка standard-changelog	15
4.20	Создание CHANGELOG	15
4.21	Коммит CHANGELOG	15
4.22	Установка Julia	16
4.23	Запуск Julia в каталоге lab01	16
4.24	Установка DrWatson	16
4.25	Инициализация проекта DrWatson	17
4.26	Установка пакетов Julia	17
4.27	Создание тестового скрипта	17
4.28	Запуск тестового скрипта	17

Список таблиц

1 Цель работы

Целью данной лабораторной работы является освоение практических навыков подготовки рабочего пространства для имитационного моделирования, настройка инструментов (Git, Git Flow, DrWatson), а также первичная реализация и документирование модели экспоненциального роста на языке Julia с использованием подхода литературного программирования.

2 Задание

1. Настроить рабочее пространство для курса в соответствии с соглашением Denote.
2. Создать и настроить репозиторий на GitHub, используя Git Flow.
3. Установить необходимое программное обеспечение (Git, Node.js, pnpm, git-flow, Julia).
4. Создать проект DrWatson для лабораторной работы.
5. Установить необходимые Julia-пакеты.
6. Реализовать модель экспоненциального роста.
7. Преобразовать скрипт в формат литературного программирования.
8. Интегрировать результаты в итоговый отчёт.

3 Теоретическое введение

3.1 Модель экспоненциального роста

Экспоненциальный рост — это процесс увеличения величины, при котором скорость роста в каждый момент времени пропорциональна текущему значению этой величины. Чем больше система, тем быстрее она растёт.

Дифференциальное уравнение, описывающее экспоненциальный рост, имеет вид:

$$\frac{du}{dt} = \alpha u$$

где: - u — текущее значение растущей величины; - t — время; - α — константа скорости роста.

Решение этого уравнения:

$$u(t) = u_0 e^{\alpha t}$$

где u_0 — начальное значение.

Ключевой характеристикой экспоненциального роста является постоянное время удвоения:

$$T_2 = \frac{\ln 2}{\alpha}$$

3.2 Инструменты

В работе используются следующие инструменты:

- **Git** — система контроля версий;
- **Git Flow** — модель ветвления для Git;
- **GitHub** — хостинг репозитория;
- **Julia** — язык программирования для научных вычислений;
- **DrWatson.jl** — фреймворк для организации научных проектов;
- **Literate.jl** — инструмент для литературного программирования;
- **Quarto** — система для создания научных отчётов.

4 Выполнение лабораторной работы

4.1 1. Установка Git

Первым шагом устанавливаем систему контроля версий Git (рис. 1).

```
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ sudo apt install git
```

Рисунок 4.1: Установка Git

4.2 2. Настройка Git

Настраиваем имя пользователя и email для Git (рис. 2).

```
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global user.name "Maria Ulitina"
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global user.email "marieulitina1@yandex.ru"
```

Рисунок 4.2: Настройка имени и email в Git

4.3 3. Базовая конфигурация Git

Выполняем базовую настройку Git: указываем имя ветки по умолчанию, настраиваем окончания строк (рис. 3).

```
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global user.name "Maria Ulitina"
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global user.email "marieulitina1@yandex.ru"
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global core.quotepath false
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global init.defaultBranch master
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global core.autocrlf input
mmulitina@mmulitina-VMware-Virtual-Platform: ~$ git config --global core.safecrlf warn
```

Рисунок 4.3: Базовая конфигурация Git

4.4 4. Создание SSH-ключа

Генерируем SSH-ключ для подключения к GitHub по протоколу SSH (рис. 4).

```
mmulitina@mmulitina-VMware-Virtual-Platform: $ ssh-keygen -t ed25519 -C "marieulitina1@yandex.ru"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/mmulitina/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/mmulitina/.ssh/id_ed25519
Your public key has been saved in /home/mmulitina/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:m3dLQJGLzX46cdxwE15WcGAPFunzvBwFZdIphKF4NJ0 marieulitina1@yandex.ru
The key's randomart image is:
+--[ED25519 256]--+
  =o+=ooB..|
  o =+o+E o.|
  ..+o..o |
  o..o... |
  S =. =. |
  *.Bo.. |
  + Bo* |
  o.=.+ |
  o++ |
+----[SHA256]-----+
mmulitina@mmulitina-VMware-Virtual-Platform: $
mmulitina@mmulitina-VMware-Virtual-Platform: $ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICqM0aFSnlwC0gVkeNfx2tegu7sj0aoP0vldIpk4CiDe marieulitina1@yandex.ru
```

Рисунок 4.4: Создание SSH-ключа

4.5 5. Установка Node.js

Пытаемся установить Node.js (рис. 5).

```
mmulitina@mmulitina-VirtualBox:~$ sudo apt-get install node.js
[sudo] password for mmulitina:
```

Рисунок 4.5: Установка Node.js

4.6 6. Установка yarn

Устанавливаем пакетный менеджер yarn (рис. 6).

```
mmulitina@mmulitina-VirtualBox:~$ sudo apt-get install yarn
```

Рисунок 4.6: Установка yarn

4.7 7. Обновление списка пакетов

Обновляем список доступных пакетов (рис. 7).

```
mmulitina@mmulitina-VirtualBox:~$ sudo apt update
```

Рисунок 4.7: Обновление списка пакетов

4.8 8. Установка nodejs и npm

Устанавливаем Node.js и npm (рис. 8).

```
mmulitina@mmulitina-VirtualBox:~$ sudo apt install nodejs npm
```

Рисунок 4.8: Установка nodejs и npm

4.9 9. Проверка версий и установка рnpm

Проверяем установленные версии Node.js и npm, устанавливаем rnpm (рис. 9).

```
mmulitina@mmulitina-VirtualBox:~$ node --version
v18.19.1
mmulitina@mmulitina-VirtualBox:~$ npm --version
9.2.0
mmulitina@mmulitina-VirtualBox:~$ sudo npm install -g rnpm
```

Рисунок 4.9: Проверка версий и установка rnpm

4.10 10. Настройка rnpm

Выполняем настройку rnpm для дальнейшего использования (рис. 10).

```
mmulitina@mmulitina-VirtualBox:~$ pnpm setup
Appended new lines to /home/mmulitina/.bashrc

Next configuration changes were made:
export PNPM_HOME="/home/mmulitina/.local/share/pnpm"
case ":$PATH:" in
  *:$PNPM_HOME:*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /home/mmulitina/.bashrc
```

Рисунок 4.10: Настройка pnpm

4.11 11. Установка Git Flow

Устанавливаем Git Flow для управления ветками в репозитории (рис. 11).

```
mmulitina@mmulitina-VirtualBox:~$ sudo apt-get install git-flow
```

Рисунок 4.11: Установка Git Flow

4.12 12. Создание структуры рабочего пространства

Создаём структуру каталогов для курса в соответствии с соглашением Denote (рис. 12).

```
mmulitina@mmulitina-VirtualBox:~$ mkdir -p ~/work/study/2026-1
mmulitina@mmulitina-VirtualBox:~$ cd ~/work/study/2026-1
```

Рисунок 4.12: Создание рабочего пространства

4.13 13. Клонирование репозитория

Клонируем созданный на GitHub репозиторий курса (рис. 13).

```
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1$ git clone git@github.com:mmulitina/2026-1-study-simulation-modeling.git
```

Рисунок 4.13: Клонирование репозитория

4.14 14. Настройка каталога курса

Инициализируем курс и выполняем подготовку структуры через Makefile (рис. 14).

```
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1$ cd 2026-1-study-simulation-modeling/  
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ echo simulation-  
modeling > COURSE  
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ make prepare
```

Рисунок 4.14: Настройка каталога курса

4.15 15. Отправка изменений на GitHub

Отправляем первоначальную структуру в удалённый репозиторий (рис. 15).

```
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git push
```

Рисунок 4.15: Push на GitHub

4.16 16. Инициализация Git Flow

Инициализируем Git Flow в репозитории (рис. 16).

```
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git flow init -f
```

Рисунок 4.16: Инициализация Git Flow

4.17 17. Проверка веток и отправка

Проверяем созданные ветки (develop и master) и отправляем их на GitHub (рис. 17).

```
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git branch  
* develop  
master  
mmulitina@mmulitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git push -u --al
```

Рисунок 4.17: Проверка веток и push

4.18 18. Создание первого релиза

Создаём релиз с версией 1.0.0 (рис. 18).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git flow release
start 1.0.0
Switched to a new branch 'release/1.0.0'

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'
```

Рисунок 4.18: Создание релиза

4.19 19. Установка standard-changelog

Устанавливаем инструмент для генерации журнала изменений (рис. 19).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ npm add -g standard-changelog
```

Рисунок 4.19: Установка standard-changelog

4.20 20. Создание CHANGELOG

Генерируем файл CHANGELOG.md для первого релиза (рис. 20).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ standard-changelog --first-release
```

Рисунок 4.20: Создание CHANGELOG

4.21 21. Коммит CHANGELOG

Добавляем CHANGELOG.md в индекс и создаём коммит (рис. 21).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git add CHANGELOG.md
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ git commit -am 'chore(site): add changelog'
[release/1.0.0 50ea4db] chore(site): add changelog
1 file changed, 9 insertions(+)
create mode 100644 CHANGELOG.md
```

Рисунок 4.21: Коммит CHANGELOG

4.22 22. Установка Julia

Устанавливаем Julia через snap-пакет (рис. 22).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ sudo snap install julia --classic
```

Рисунок 4.22: Установка Julia

4.23 23. Запуск Julia

Подтверждаем установку Julia и переходим в каталог лабораторной работы (рис. 23).

```
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ sudo snap install julia --classic
[sudo] password for mmultitina:
julia 1.12.4 from The Julia Language (julia-lang) installed
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling$ cd labs
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling/labs$ cd lab01
mmultitina@mmultitina-VirtualBox:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01$ julia

Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.
Version 1.12.4 (2026-01-06)
Official https://julialang.org release

julia>
```

Рисунок 4.23: Запуск Julia в каталоге lab01

4.24 24. Установка DrWatson

В Julia устанавливаем пакет DrWatson (рис. 24).

```
julia> using Pkg
Pkg>
julia> Pkg.add("DrWatson")
Installing known registries into `~/.julia`
```

Рисунок 4.24: Установка DrWatson

4.25 25. Инициализация проекта DrWatson

Создаём структуру проекта DrWatson с именем «project» (рис. 25).


```
julia> initialize_project("project", authors= "Maria Ulitina", git=false)
Activating new project at `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project`
Resolving package versions...
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project/Manifest.toml`
```

Рисунок 4.25: Инициализация проекта DrWatson

4.26 26. Установка пакетов Julia

Устанавливаем все необходимые пакеты для работы (рис. 26).

```
mmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01 $ julia --project=. -e 'using Pkg; Pkg.add([DrWatson, "DifferentialEquations", "Plots", "DataFrames", "CSV", "JLOps", "Iterate", "Tullio", "BenchmarkTools"]);'
```

Рисунок 4.26: Установка пакетов Julia

4.27 27. Создание тестового скрипта

Создаём файл test_setup.jl для проверки установки (рис. 27).

```
mmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01 $ touch test_setup.jl
mmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01 $ cd project
mmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project $ cd scripts
mmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project/scripts $ julia test_setup.jl
```

Рисунок 4.27: Создание тестового скрипта

4.28 28. Запуск тестового скрипта

Запускаем тестовый скрипт для проверки установки пакетов (рис. 28).

```
^C^C^Cmmulitina@mmulitina-VirtualBox: ~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project $ julia --project=. scripts/test_setup.jl
① OrdinaryDiffEqFIRK
✅ Проект активирован: /home/mmultipina/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab01/project
Проверка пакетов:
✓ DrWatson
```

Рисунок 4.28: Запуск тестового скрипта

4.29 30. Реализация модели экспоненциального роста

Далее представлен код модели экспоненциального роста, реализованный в стиле литературного программирования. Этот код был создан в файле `scripts/02_exponential_growth.jl` и автоматически преобразован с помощью `Literate.jl` в формат Quarto.

5 Выводы

В ходе выполнения лабораторной работы было подготовлено рабочее пространство для курса имитационного моделирования, выполнена настройка всех необходимых инструментов:

- Установлены и настроены Git, Git Flow, создан SSH-ключ для подключения к GitHub;
- Установлены Node.js, npm, yarn, pnpm для работы с инструментами форматирования коммитов;
- Установлен и настроен язык программирования Julia;
- Создан проект DrWatson и установлены необходимые пакеты;
- Реализована и задокументирована в стиле литературного программирования модель экспоненциального роста.

Полученные навыки являются основой для дальнейшего изучения различных подходов к имитационному моделированию: системно-динамического, агентного, дискретно-событийного и сетей Петри.

Список литературы